```
import javax.swing.*;
import java.awt.*;

public class Main extends JFrame {

    String[] foods = {
        "Masala Fries", "Paneer Wrap", "Chilli Paneer",
        "Veg Burger", "Paneer Pizza",
        "Chicken Burger", "Chicken Biryani", "Chicken 65",
        "Grilled Chicken", "Chicken Noodles",
        "Brownie", "Chocolate Cake", "Ice Cream",
        "Gulab Jamun", "Chocolate Milkshake",
        "Lemon Juice", "Watermelon Juice", "Mango Juice",
        "Iced Tea", "Fresh Lime Soda",
        "Veg Fried Rice", "Tomato Soup", "Pasta",
        "Veg Sandwich", "Mushroom Pizza"
    };

    int[] prices = {
        60, 100, 120, 80, 140,
        120, 180, 150, 200, 130,
        60, 90, 50, 40, 90,
        40, 50, 60, 50, 45,
        110, 70, 120, 70, 150
    };

    String[] types = {
        "Veg", "Veg", "Veg", "Veg", "Veg",
        "Non-Veg", "Non-Veg", "Non-Veg", "Non-Veg", "Non-Veg",
        "Veg", "Veg", "Veg", "Veg", "Veg",
        "Veg", "Veg", "Veg", "Veg", "Veg",
        "Veg", "Veg", "Veg", "Veg", "Veg"
    };

    String[] moods = {
        "Spicy", "Spicy", "Spicy", "Comfort", "Comfort",
        "Spicy", "Comfort", "Spicy", "Comfort", "Spicy",
        "Sweet", "Sweet", "Sweet", "Sweet", "Sweet",
        "Refreshing", "Refreshing", "Refreshing",
        "Refreshing", "Refreshing",
        "Comfort", "Comfort", "Comfort", "Comfort", "Comfort"
    };

    int[] cart = new int[foods.length];

    JComboBox<String> preference = new JComboBox<>(
        new String[]{"All", "Veg", "Non-Veg"});
```

```
JComboBox<String> mood = new JComboBox<>(
    new String[]{"All", "Spicy", "Sweet", "Refreshing", "Comfort"});

JTextField budget = new JTextField("300", 10);
JTextArea output = new JTextArea();

Color purple = new Color(104, 76, 160);
Color lavender = new Color(244, 239, 255);
Color darkPurple = new Color(65, 45, 95);
Color palePurple = new Color(232, 220, 250);

Main() {
    setTitle("Smart Food Ordering System");
    setSize(1000, 700);
    setMinimumSize(new Dimension(800, 600));
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    getContentPane().setBackground(lavender);
    setLayout(new BorderLayout(10, 10));

    JLabel title = new JLabel(
        "SMART FOOD ORDERING SYSTEM", JLabel.CENTER);
    title.setFont(new Font("SansSerif", Font.BOLD, 25));
    title.setForeground(Color.WHITE);
    title.setOpaque(true);
    title.setBackground(purple);
    title.setBorder(BorderFactory.createEmptyBorder(18, 10, 18, 10));
    add(title, BorderLayout.NORTH);

    // Left-side controls
    JPanel sidebar = new JPanel();
    sidebar.setLayout(new BoxLayout(sidebar, BoxLayout.Y_AXIS));
    sidebar.setBackground(lavender);
    sidebar.setBorder(BorderFactory.createEmptyBorder(12, 12, 12, 8));
    sidebar.setPreferredSize(new Dimension(230, 0));

    JLabel filterTitle = new JLabel("FOOD FILTERS");
    filterTitle.setFont(new Font("SansSerif", Font.BOLD, 17));
    filterTitle.setForeground(darkPurple);
    filterTitle.setAlignmentX(Component.LEFT_ALIGNMENT);

    sidebar.add(filterTitle);
    sidebar.add(Box.createVerticalStrut(15));

    sidebar.add(new JLabel("Food Preference"));
    preference.setMaximumSize(new Dimension(200, 32));
    sidebar.add(preference);
    sidebar.add(Box.createVerticalStrut(12));
```

```
sidebar.add(new JLabel("Choose Your Mood"));
mood.setMaximumSize(new Dimension(200, 32));
sidebar.add(mood);
sidebar.add(Box.createVerticalStrut(12));

sidebar.add(new JLabel("Your Budget (Rs.)"));
budget.setMaximumSize(new Dimension(200, 32));
sidebar.add(budget);
sidebar.add(Box.createVerticalStrut(20));

addSideButton(sidebar, "View Full Menu", e -> showMenu());
addSideButton(sidebar, "Recommend Food", e -> recommendFood());
addSideButton(sidebar, "Budget Suggestions", e -> showBudget());
addSideButton(sidebar, "Add Food to Cart", e -> addFood());
addSideButton(sidebar, "View Bill", e -> showBill());
addSideButton(sidebar, "Clear Cart", e -> clearCart());

sidebar.add(Box.createVerticalStrut(15));

JLabel tip = new JLabel(
    "<html><div style='width:170px'>"
    + "Tip: Choose your preference and mood "
    + "before getting recommendations.</div></html>");
tip.setForeground(darkPurple);
sidebar.add(tip);

add(sidebar, BorderLayout.WEST);

// Main display area
output.setEditable(false);
output.setFont(new Font("Monospaced", Font.PLAIN, 15));
output.setForeground(darkPurple);
output.setBackground(Color.WHITE);
output.setMargin(new Insets(15, 15, 15, 15));
output.setLineWrap(true);
output.setWrapStyleWord(true);

JScrollPane scroll = new JScrollPane(output);
scroll.setBorder(BorderFactory.createTitledBorder(
    BorderFactory.createLineBorder(palePurple, 2),
    "MENU AND ORDER DETAILS"));
add(scroll, BorderLayout.CENTER);

JLabel footer = new JLabel(
    "Fresh Choices  |  Smart Budget  |  Easy Ordering",
    JLabel.CENTER);
footer.setFont(new Font("SansSerif", Font.BOLD, 14));
```

```
    footer.setForeground(purple);
    footer.setBorder(BorderFactory.createEmptyBorder(10, 5, 10, 5));
    add(footer, BorderLayout.SOUTH);

    // Update recommendations when filters change
    preference.addActionListener(e -> recommendFood());
    mood.addActionListener(e -> recommendFood());

    showMenu();
    setVisible(true);
}

void addSideButton(JPanel panel, String text,
                   java.awt.event.ActionListener action) {
    JButton button = new JButton(text);
    button.setFont(new Font("SansSerif", Font.BOLD, 13));
    button.setBackground(purple);
    button.setForeground(Color.WHITE);
    button.setFocusPainted(false);
    button.setAlignmentX(Component.LEFT_ALIGNMENT);
    button.setMaximumSize(new Dimension(205, 40));
    button.setPreferredSize(new Dimension(205, 40));
    button.addActionListener(action);

    panel.add(button);
    panel.add(Box.createVerticalStrut(10));
}

boolean matches(int i) {
    String p = (String) preference.getSelectedItem();
    String m = (String) mood.getSelectedItem();

    return (p.equals("All") || types[i].equals(p))
        && (m.equals("All") || moods[i].equals(m));
}

void showMenu() {
    output.setText("               COMPLETE FOOD MENU\n");
    output.append("================================================\n\n");

    String[] categories = {
        "Spicy", "Sweet", "Refreshing", "Comfort"
    };

    for (String category : categories) {
        output.append(">>> " + category.toUpperCase() + " FOODS\n");
        output.append("------------------------------------------\n");
```

```
            for (int i = 0; i < foods.length; i++) {
                if (moods[i].equals(category)) {
                    output.append(String.format(
                        "%-23s %-9s Rs.%d%n",
                        foods[i], types[i], prices[i]));
                }
            }
            output.append("\n");
        }
    }

    void recommendFood() {
        output.setText("             PERSONALIZED RECOMMENDATIONS\n");
        output.append("================================================\n\n");

        boolean found = false;

        for (int i = 0; i < foods.length; i++) {
            if (matches(i)) {
                output.append(String.format(
                    "%-23s %-9s Rs.%d%n",
                    foods[i], types[i], prices[i]));
                found = true;
            }
        }

        if (!found) {
            output.append("No matching foods found.\n");
        }
    }

    void showBudget() {
        try {
            int limit = Integer.parseInt(budget.getText().trim());

            if (limit < 0) {
                JOptionPane.showMessageDialog(this,
                    "Budget cannot be negative.");
                return;
            }

            output.setText("             SMART BUDGET SUGGESTIONS\n");

output.append("================================================\n\n");
            output.append("Your budget: Rs." + limit + "\n\n");

            boolean found = false;
```

```
            for (int i = 0; i < foods.length; i++) {
                if (matches(i) && prices[i] <= limit) {
                    output.append(String.format(
                        "%-23s Rs.%d%n", foods[i], prices[i]));
                    found = true;
                }
            }

            if (!found) {
                output.append("No food matches these filters and budget.\n");
                output.append("Increase your budget or change your filters.\n");
            }
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this,
                "Please enter a valid budget.");
        }
    }

    void addFood() {
        String[] choices = new String[foods.length];
        int count = 0;

        for (int i = 0; i < foods.length; i++) {
            if (matches(i)) {
                choices[count++] = foods[i] + " - Rs." + prices[i];
            }
        }

        if (count == 0) {
            JOptionPane.showMessageDialog(this,
                "No matching food. Change your filters.");
            return;
        }

        String[] available = new String[count];
        System.arraycopy(choices, 0, available, 0, count);

        String selected = (String) JOptionPane.showInputDialog(
            this, "Choose a food:", "Add to Cart",
            JOptionPane.QUESTION_MESSAGE, null,
            available, available[0]);

        if (selected == null) return;

        for (int i = 0; i < foods.length; i++) {
            if (selected.equals(foods[i] + " - Rs." + prices[i])) {
                String text = JOptionPane.showInputDialog(
                    this, "Enter quantity:");
```

```
            if (text == null) return;

            try {
                int quantity = Integer.parseInt(text.trim());

                if (quantity <= 0) {
                    JOptionPane.showMessageDialog(this,
                        "Quantity must be greater than zero.");
                    return;
                }

                cart[i] += quantity;
                JOptionPane.showMessageDialog(this,
                    foods[i] + " added successfully!");
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(this,
                    "Enter a valid whole number.");
            }
            return;
        }
    }
}

void showBill() {
    output.setText("                    YOUR BILL\n");
    output.append("================================================\n\n");

    int total = 0;

    for (int i = 0; i < foods.length; i++) {
        if (cart[i] > 0) {
            int amount = cart[i] * prices[i];
            output.append(foods[i] + " x " + cart[i]
                + " = Rs." + amount + "\n");
            total += amount;
        }
    }

    if (total == 0) {
        output.append("Your cart is empty.\n");
        return;
    }

    output.append("\n------------------------------------------\n");
    output.append("Subtotal: Rs." + total + "\n");

    if (total >= 300) {
```

```
            int discount = total / 10;
            output.append("10% Discount: Rs." + discount + "\n");
            total -= discount;
        } else {
            output.append("Discount: Rs.0\n");
            output.append("Spend Rs.300 to qualify for 10% off.\n");
        }

        output.append("-----------------------------------------------\n");
        output.append("FINAL BILL: Rs." + total + "\n");
    }

    void clearCart() {
        for (int i = 0; i < cart.length; i++) {
            cart[i] = 0;
        }

        output.setText("Your cart has been cleared.\n");
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Main());
    }
}
```